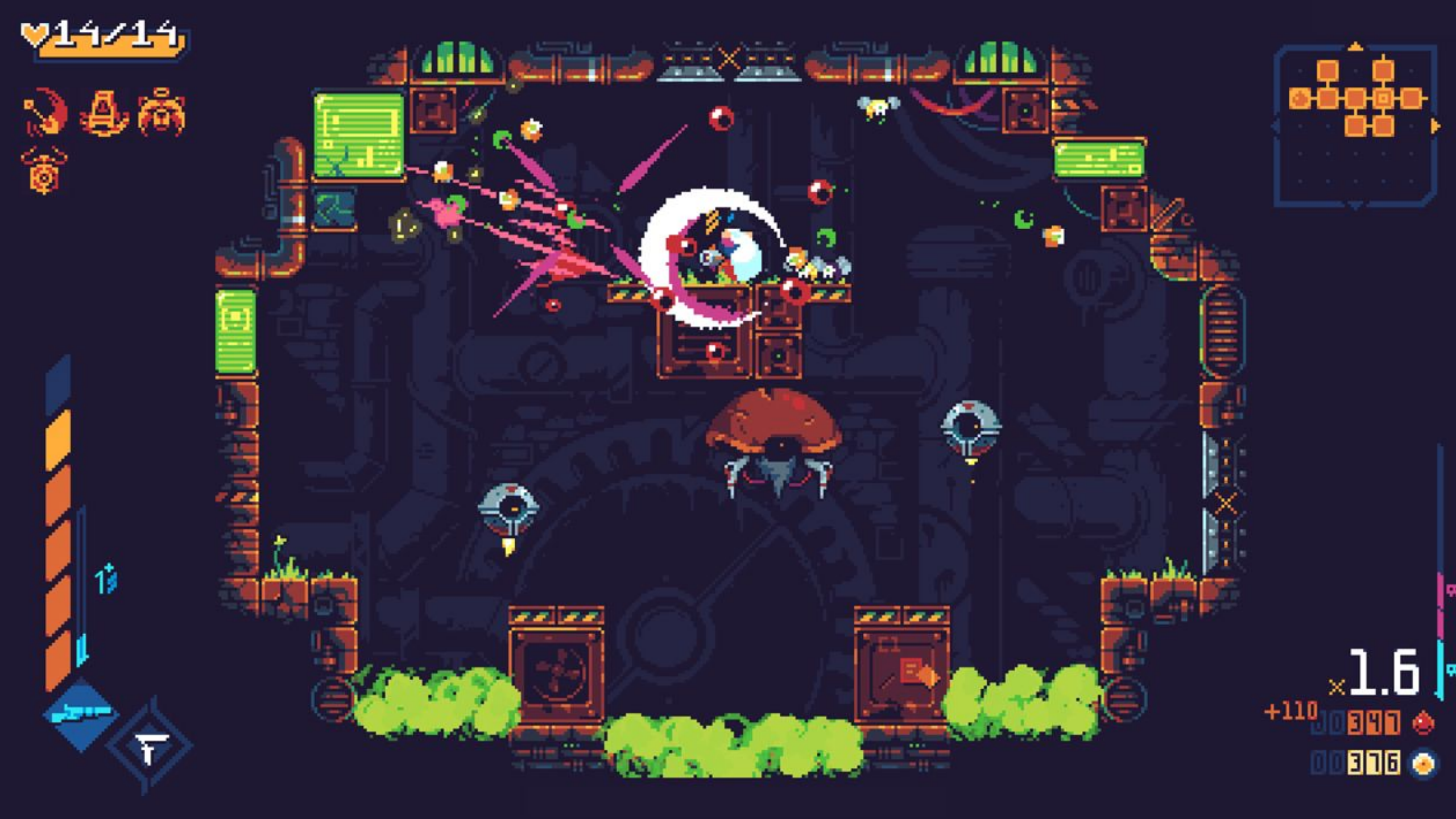
Generador de Masmorres

Programació pas a pas en C# per a Unity

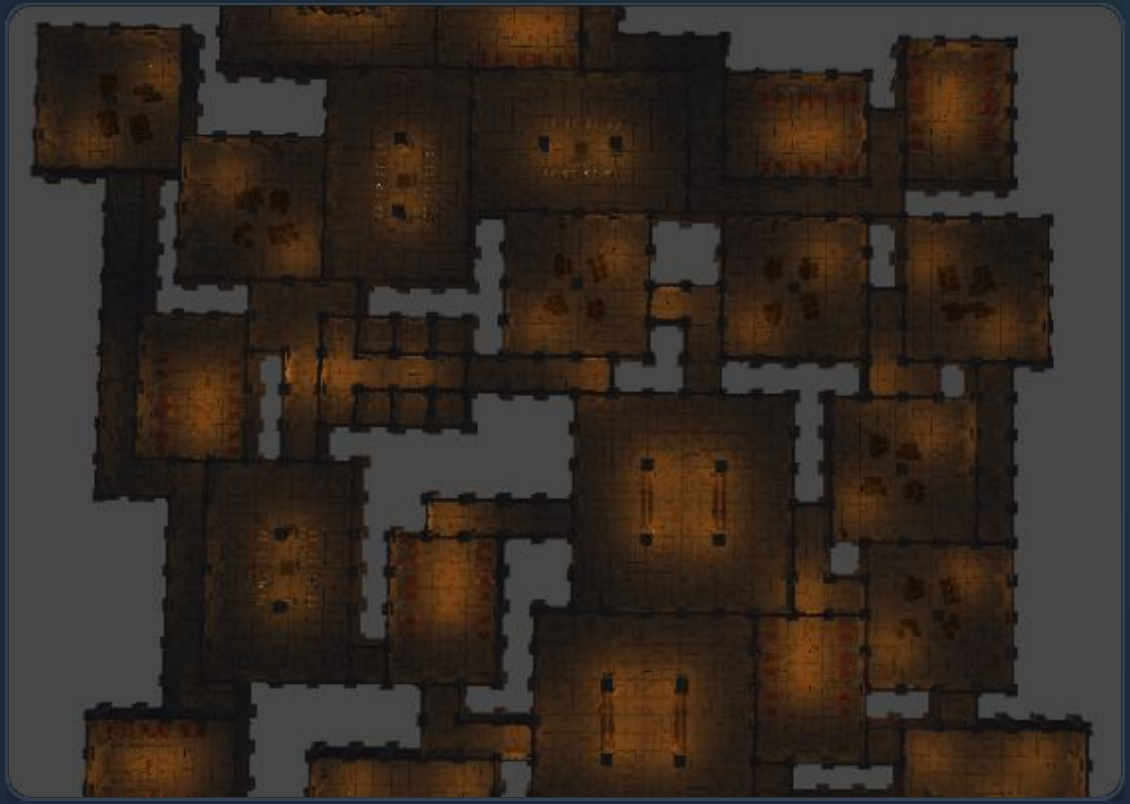
00
01
01
1.00
2.73
3.50
6.50
1.00
0.00
0.0%
0.0%

+1 Hierba 19
Work In Progress Gameplay


14/14
1.6
+110
341
376

Algoritme: Visió General

- Crear un grid buit per guardar la masmorra.
- Crear la sala inicial al centre del grid.
- Afegir-la a una cua de sales a processar.
- Mentre no s'arribi al nombre de sales:
  - Agafar sala de la cua i guardar al grid.
  - Buscar veïns disponibles i crear-los aleatòriament.
  - Afegir nous veïns a la cua.
- Connectar les sales i instanciar el prefab.



Projecte base

- Accedirem a la següent URL i ens baixarem el projecte base.
  https://github.com/edu1010/ProceduralGame

1. Estructura de Dades: Room.cs

```
public class Room {
    private Vector2Int coord;
    public Dictionary Neighbours;

    public List getNeighbourCoordinates() {
        // Retorna N, S, E, W del voltant
    }
}
```

La base del sistema

Abans de programar el generador, necessitem una classe Room que guardi:

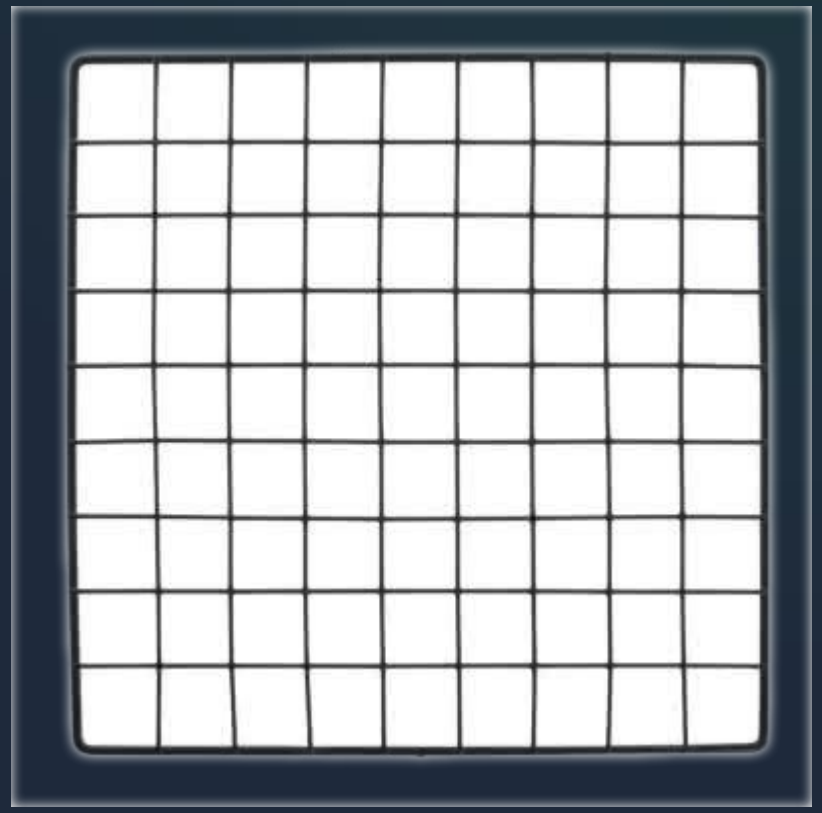
- **Coordenades:** Posició relativa al grid.
- **Diccionari de Veïns:** Guarda quina sala hi ha a cada direcció cardinal.

2. El Generador: Inicialització

El script principal gestiona la lògica de creació dins de Unity:

```
void InitGrid() {
    int gridSize = 3 * _numberOfRooms;
    Grid = new Room[gridSize, gridSize];
}
```

Definim un **Grid** prou gran perquè la masmorra pugui créixer cap a qualsevol direcció sense desbordar la matriu.
També aquí carreguem tots el tipus de sales per poder-la crear després.



3. Pas a pas: L'Expansió (BFS)

Enqueue

Comencem amb la sala central i la posem a la cua (Queue).

```
1 reference
void GenerateDungeon()
{
    InitGrid();
    // Centra la primera sala al mig del grid.
    Vector2Int coord = new Vector2Int(Grid.GetLength(0) / 2 - 1, Grid.GetLength(1) / 2 - 1);
    Queue<Room> roomsToCreate = new Queue<Room>();

    Room firstRoom = new Room(coord);
    roomsToCreate.Enqueue(firstRoom);

    // Llista de sales creades.
    List<Room> createdRooms = new List<Room>();
    //createdRooms.Add(firstRoom);

    AddRooms(roomsToCreate, createdRooms);
    ConnectNeighbours(createdRooms);
}
```

Dequeue

Treiem la sala de la cua per processar els seus espais buits.

Aleatorietat

Triem quants i quins veïns crear per fer el mapa impredictible.

3. Pas a pas: L'Expansió (BFS)

AddRoms al principi només té una sala, però cada vegada que es processa AddNeighbours, s'afegeixen entre 1 a 4 sales al voltant de la que estem processant de manera aleatòria.

```
1 referencia
private void AddRooms(Queue<Room> roomsToCreate, List<Room> createdRooms)
{
    while(roomsToCreate.Count>0 && createdRooms.Count < _numberOfRooms)
    {
        // Treu la primera sala de la cua per processar-la.
        Room currentRoom = roomsToCreate.Dequeue();
        // Marca la sala al grid per consultar veïns després.
        Grid[currentRoom.x, currentRoom.y] = currentRoom;
        createdRooms.Add(currentRoom);
        AddNeighbours(currentRoom, roomsToCreate);
    }
}
```

```
private void AddNeighbours(Room currentRoom, Queue<Room> roomsToCreate)
{
    List<Vector2Int> availableNeighbors = GetAvalibleNeighbors(currentRoom);

    int numberOfNeighbors = (int)UnityEngine.Random.Range(1, availableNeighbors.Count);

    CreateNeighbors(availableNeighbors, numberOfNeighbors, roomsToCreate);
}
```

3. Pas a pas: L'Expansió (BFS)

Abans de crear les sales veïnes haurem de mirar sí al grid ja hem creat en aquesta posició un altre sala.

```
private void AddNeighbours(Room currentRoom, Queue<Room> roomsToCreate)
{
    List<Vector2Int> availableNeighbors = GetAvalibleNeighbors(currentRoom);

    int numberOfNeighbors = (int)UnityEngine.Random.Range(1, availableNeighbors.Count);

    CreateNeighbors(availableNeighbors, numberOfNeighbors, roomsToCreate);
}
```

```
// Retorna coordenades de veïns disponibles (encara no creats).
1 referencia
private List<Vector2Int> GetAvalibleNeighbors(Room currentRoom)
{
    // Totes les coordenades candidates al voltant.
    List<Vector2Int> possibleNeighboursCoords = currentRoom.getNeighbourCoordinates();
    // Filtra les que encara no existeixen al grid.
    List<Vector2Int> availableNeighboursCoords = new List<Vector2Int>();
    foreach(Vector2Int coordinate in possibleNeighboursCoords)
    {
        if(this.Grid[coordinate.x,coordinate.y] == null)
        {
            availableNeighboursCoords.Add(coordinate);
        }
    }
    return availableNeighboursCoords;
}
```

3. Detectar espais lliures

```
private List GetAvalibleNeighbors(Room currentRoom) {
    var coords = currentRoom.getNeighbourCoordinates();
    foreach(var coord in coords) {
        if(this.Grid[coord.x, coord.y] == null) {
            // Aquest lloc està lliure!
        }
    }
}
```

L'algoritme de filtrat

El mètode GetAvalibleNeighbors és crucial:

- Mira les 4 coordenades adjacents.
- Només afegeix les que estan marcades com a null al grid.
- Això evita que les sales se superposin.

3. Pas a pas: L'Expansió (BFS)

Ara amb les coordenades del grid que encara estan lliures podem crear els diferents veïns. I afegim aquestes sales noves a la cua.

```
private void AddNeighbours(Room currentRoom, Queue<Room> roomsToCreate)
{
    List<Vector2Int> availableNeighbors = GetAvalibleNeighbors(currentRoom);

    int numberOfNeighbors = (int)UnityEngine.Random.Range(1, availableNeighbors.Count);

    CreateNeighbors(availableNeighbors, numberOfNeighbors, roomsToCreate);
}
```

```
1 referencia
private void CreateNeighbors(List<Vector2Int> availableNeighbors, int numberOfNeighbors, Queue<Room> roomsToCreate)
{
    // Afegeix sales veïnes aleatòries a la cua.
    for(int i = 0; i < numberOfNeighbors; i++)
    {
        int chosen = Random.Range(0, availableNeighbors.Count);
        Vector2Int choosenNeighbour = availableNeighbors[chosen];
        roomsToCreate.Enqueue( new Room(choosenNeighbour));
        availableNeighbors.Remove(choosenNeighbour);
```

```
private void AddRooms(Queue<Room> roomsToCreate, List<Room> createdRooms)
{
    while(roomsToCreate.Count>0 && createdRooms.Count < _numberOfRooms)
    {
        // Treu la primera sala de la c
        Room currentRoom = roomsToCre
        // Marca la sala al grid per co
        Grid[currentRoom.x, currentRoom
        createdRooms.Add(currentRoom);
        AddNeighbours(currentRoom, roomsToCreate);
    }
}
```

Torna a ser true si hem pogut afegir veïns.

3. Connectar les sales (Portes)

El sistema de vincle

Un cop hem col·locat totes les sales, hem d'establir els vincles direccionals:

- Recorrem la llista de sales creades.
- Si una sala té un veí real al grid, cridem a room.Connect(neighbor).
- El codi dedueix si el veí és al Nord, Sud, Est o Oest.

```
internal void Connect(Room neighbor) {
    if(neighbor.y < this.y) direction = directions.S;
    if(neighbor.y > this.y) direction = directions.N;
    // ... i així per a totes les direccions
    Neighbours.Add(direction, neighbor);
}
```

3. Guardar els veïns de cada sala.

Cada room emmagatzema en quines direccions hi té de veí.

Això ho farem servir per crear la versió “física/instanciada” després.

```
private void ConnectNeighbours(List<Room> createdRooms)
{
    foreach(var room in createdRooms)
    {
        //Cada room pot retornar les cordenades del grid del seu voltant
        List<Vector2Int> neighborCoordinates = room.getNeighbourCoordinates();
        foreach(Vector2Int coordinate in neighborCoordinates)
        {
            Room neighbor = this.Grid[coordinate.x, coordinate.y];
            if(neighbor != null)
            {
                // Connecta amb els veïns existents (1 a 4 possibles).
                room.Connect(neighbor);
            }
        }
    }
}
```

4. Generem les sales al món de unity

Amb tota la informació que hem recollit, podem recorre el grid i crear les sales necessàries. Com cada room sap els seus veïns podem fer servir aquesta informació per crear els correctes.

```
1 reference
void ShowDungeon()
{
    for (int i = 0; i < Grid.GetLength(0); i++)
    {
        for (int j = 0; j < Grid.GetLength(1); j++)
        {
            if (Grid[i, j] != null)
            {
                CreateRoom(Grid[i, j]);
            }
        }
    }
}
```

```
public void CreateRoom(Room actualRoom)
{
    bool n = false, s = false, e = false, w = false;
    foreach (directions dir in actualRoom.Neighbours.Keys)
    {
        switch (dir)
        {
            case directions.N:
                n = true;
                break;
            case directions.S:
                s = true;
                break;
            case directions.E:
                e = true;
                break;
            case directions.W:
                w = true;
                break;
            default:
                break;
        }
    }
    GameObject room = Instantiate(avalibleRooms.GetRoom(n, s, e, w), new Vector2(actualR
    room.transform.parent = _parent.transform;
}
```

4. Generem les sales al món de unity

Com cada room sap els seus veïns podem fer servir aquesta informació per crear els correctes.

Quan creem la versió física com mesura 10 metres i cada sala és quadrada en aquest cas, multipliquem per 10.

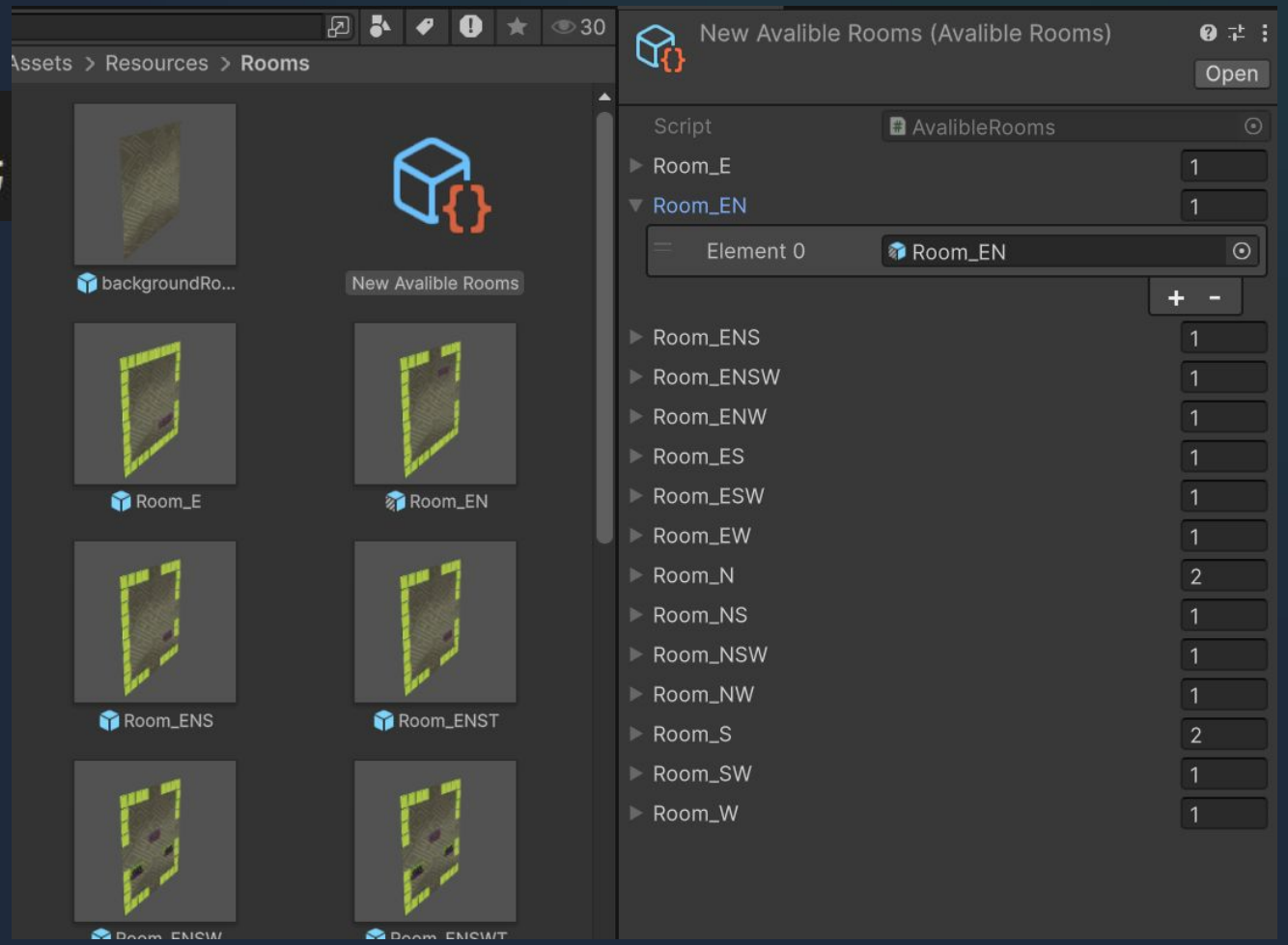
```
public void CreateRoom(Room actualRoom)
{
    bool n = false, s = false, e = false, w = false;
    foreach (directions dir in actualRoom.Neighbours.Keys)
    {
        switch (dir)
        {
            case directions.N:
                n = true;
                break;
            case directions.S:
                s = true;
                break;
            case directions.E:
                e = true;
                break;
            case directions.W:
                w = true;
                break;
            default:
                break;
        }
    }
    GameObject room = Instantiate(avalibleRooms.GetRoom(n, s, e, w), new Vector2(actualRoom.x * 10, actualRoom.y * 10), Quaternion.identity);
    room.transform.parent = _parent.transform;
}
```

```
return Room_ENSW[Random.Range(0, Room_ENSW.Length)];
```

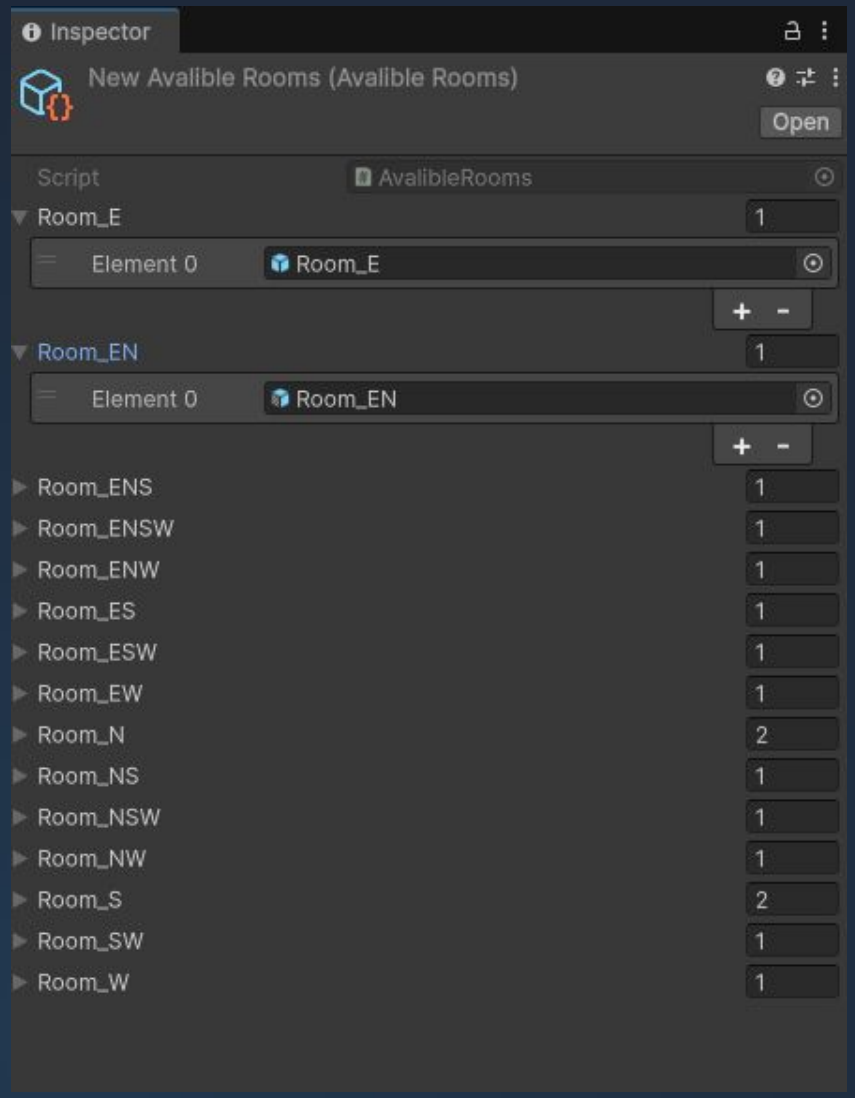
4. Generem les sales al món de unity

Per cada tipus de sala, tenim un array, enviem una sala aleatòria d'aquet tipus, el que ens permet tindre més varietat de sales.

```
return Room_ENSW[Random.Range(0, Room_ENSW.Length)];
```



6. AvalibleRooms: La Base de Dades



Gestió de Prefabs

Utilitzem un ScriptableObject per emmagatzemar tots els models de sala:

- Arrays per a cada configuració (N, NS, NSEW, etc.).
- Permet als dissenyadors afegir varietat visual sense tocar el codi.
- Selecciona un prefab aleatori dins de la categoria correcta.

7. Lògica de Selecció: GetRoom

Input (Booleans)	N, S, E, W	Codi Retornat
true, true, false, false	Nord + Sud	Room_NS[Random]
true, false, true, true	N + E + W	Room_ENW[Random]
false, true, false, true	Sud + Oest	Room_SW[Random]

El mètode fa servir una cascada d'if per determinar quina combinació de portes és la correcta.

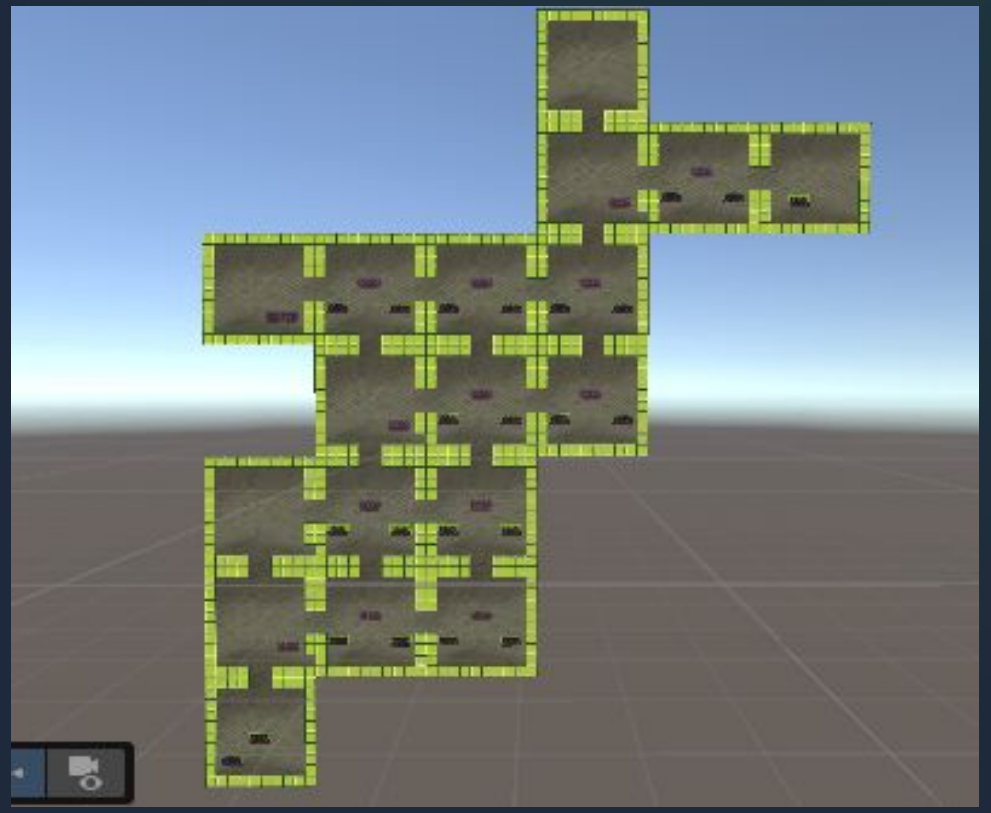
8. Instanciació: ShowDungeon

De dades a 2D

Finalment, posem els objectes a l'escena:

```
Instantiate(
    avalibleRooms.GetRoom(n, s, e, w),
    new Vector2(actualRoom.x * 10, actualRoom.y * 10),
    Quaternion.identity
);
```

Multipliquem les coordenades x10 per donar espai físic a cada sala (assumint que cada sala fa 10 unitats).



Resum del Flux de Treball

Dades

Creació de la classe Room i Enum de direccions.

Algoritme

Generació de coordenades al Grid usant una Cua.

Lògica

Càlcul de connexions entre sales veïnes.

Visual

Instanciació dels prefabs segons le portes.

Preguntes?

Gràcies per la vostra atenció

```
// Feliç codificació!
```